



Лабораторийн ажил 9

Ажилтан бүртгэлийн систем

Мэдээлэл, Компьютерын Ухааны тэнхим

Хичээлийн нэр (индекс): Объект хандлагат программчлал (ISCI201)

Лабораторийн хуваарь: Пүрэв гариг 07:40 – 10:05 цаг

Суралцагчдийн нэр: Э.Баттамир, Ц.Сумъяабазар, Г.Билгүүн

Багшийн нэр: М.Золжаргал (Ph.D)

Удамшил ямар харьцаа үүсгэдэг вэ?

Удамшил нь объект хандлагат программчлалын хамгийн чухал зарчмуудын нэг бөгөөд:

- “is-a” relationship буюу “... бол ...” гэсэн харилцааг илэрхийлдэг.

Жишээ: Employee is a Person, Circle is a Shape.

Удамшлын гол зорилго бол:

- кодын ахин ашиглалт
- логикийн давхардлыг багасгах
- олон хэлбэржилт (polymorphism) ашиглах боломжийг олгох юм

Объект хандлагат программчлалд хэрхэн хэрэгжүүлдэг вэ?

```
class Person {  
    //...  
};
```

```
class Employee : public Person {  
    // Employee нь Person-ы шинж чанарыг өвлөнө  
    private:  
        string companyId;  
        string title;  
        Date startDate;  
        vector<JobDescription> jobDescriptions;  
        vector<Child> children;  
        Spouse spouse;  
        Division division;
```

Бүрдэл харьцаа гэж юу вэ?

Бүрдэл нь:

- “has-a” буюу “... нь ...-тай” гэсэн харилцаа.
- Өндөр түвшний нягт холбоотой харилцаа.
- Эцэг объект уствал доторх объектууд хамт устдаг.

Хоёр объектын хоорондын харилцаа бөгөөд эцэг объектын амьдралын мөчлөг нь хүүхэд объектын амьдралын мөчлөгийг бүрэн тодорхойлдог. Эцэг объект устах үед түүнд багтсан (composed) бүх объектууд хамт устдаг.

Жишээ:

- Employee has a Spouse
- Car has an Engine

Объект хандлагат программчлалд хэрхэн хэрэгжүүлдэг вэ?

```
class Employee : public Person {  
    private:  
        string companyId;  
        string title;  
        Date startDate;  
        vector<JobDescription>  
jobDescriptions;  
        vector<Child> children;  
        Spouse spouse;  
        Division division;  
}
```

Aggregation харьцааны тухай ойлголт

Тодорхойлолт

Aggregation бол OOP-ийн "HAS-A" харьцаа бөгөөд нэг объект нөгөө объектыг агуулдаг боловч хоёулаа бие даан оршин тогтнох боломжтой.

Үндсэн шинж чанарууд

- Сул холболт (Weak ownership)
- Агуулагдах объект бие даан амьдарч чадна
- Composition-оос ялгаатай: устгахад хамт устдаггүй
- UML: ромбтой суман
- C++-д заалт (pointer) эсвэл reference ашигладаг

```
class Employee : public Person {  
    private:  
        Division division;  
    public:  
        void setDivision(Division division);  
};
```

Бүрдэл харьцааны хориглолт (constrain) гэж юу вэ?

Тодорхойлолт:

Объектуудын хоорондын харьцаанд тавигдах
дүрэм, хязгаарлалт бөгөөд холбоо зөв, логиктой байхыг хангана.

- Employee нь Division-тэй заавал холбоотой байх
- Division хоосон (null) байж болохгүй
- Нэг Employee нэг л Division-д харьяалагдана

Хориглолт нь системд алдаатай өгөгдөл орохоос сэргийлж, мэдээллийн үнэн зөв байдлыг хадгална.

```
class Employee : public Person {  
    private:  
        // 1. Агрегаци харьцаа  
        vector<Child> children;  
  
        // 2. Вектор ашигласан бүрдэл  
        Division division;  
    public:  
        // 3. Хориглолт  
        void setDivision(Division division);  
};
```

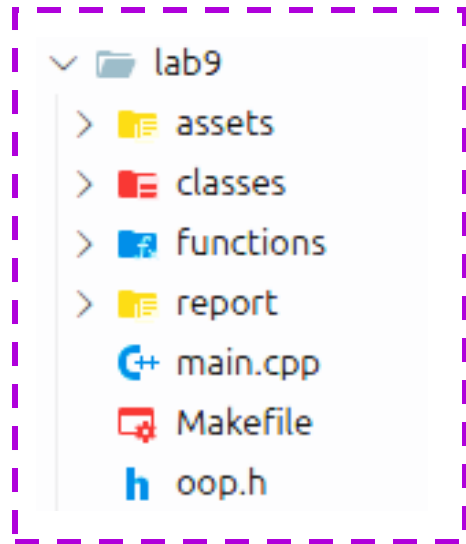
Хэрэгжүүлэлт: Кодын стандарт

Объект хандалтат программчлал

Лабораторийн ажил хийх заавар

1. Зөвхөн өөрийн оноогдсон фолдерт өөрчлөлт оруулна.
2. Хувьсагч, функц, класс, struct зэргийг англи үгээр тодорхойлно. Нэр нь утга санаагаа бүрэн илэрхийлсэн, тодорхой байх ёстой.
3. Бичиглэлийн стандарт :
 - Классын нэр: `PascalCase`
 - Классын метод болон функцүүд: `camelCase`
 - Local хувьсагчид: `camelCase`
 - Global хувьсагчид: `STREAMING_SNAKE` (Жишээ нь: `GLOBAL_VARIABLE_NAME`)
4. Фолдер болон файлын нэр: `camelCase` хэлбэрээр бичнэ.
5. Тайлан болон илтгэлийг `report` фолдероос авна.
6. Хөрвүүлэлт: `make` команд ашиглан лабораторийн ажлыг машины код руу хөрвүүлж нэгтгэнэ.
7. Тайлан болон үзүүлэнд кодыг оруулахдаа `vsCode` ийн theme ийг `Light2026` болгож `copy paste` хийх байдлаар оруулна.

Хэрэгжүүлэлт: Модульчлах программчлал



classes

person

employee

child

division

spouse

jobDescription

functions

printObj

setObj

Хэрэгжүүлэлт: Person

```
class Person {  
private:  
    string name;  
    string ssNum;  
    int age;  
public:  
    Person();  
    Person(string name, string ssNum, int age);  
    string getName() const;  
    string getSSNum() const;  
    int getAge() const;  
    void setName(string name);  
    void setSSNum(string ssNum);  
    void setAge(int age);  
    void print() const;  
};
```

Хэрэгжүүлэлт: Employee

```
class Employee : public Person {  
    private:  
        string companyId;  
        string title;  
        Date startDate;  
        vector<JobDescription> jobDescriptions;  
        vector<Child> children;  
        Spouse spouse;  
        Division division;
```

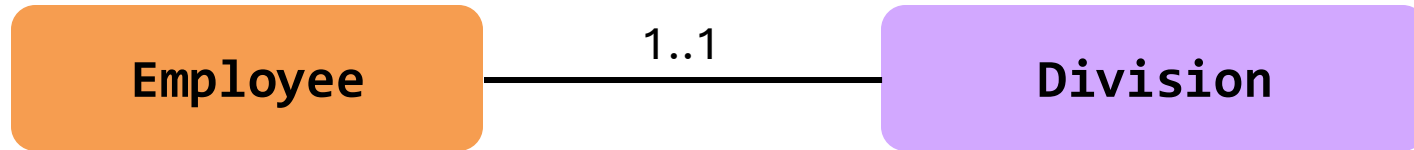
Хэрэгжүүлэлт: Division, JobDescription

```
while (true)
{
    cout << "\n1. Ажилчин нэмэх    0. Гаргах" << endl;
    cout << "Сонголт: ";
    if (!(cin >> add)) {
        cin.clear(); cin.ignore(1000, '\n'); continue;
    }

    if (add == 0) break;

    if (add == 1)
    {
        string name, ssNum, id, title, jobName, divName;
        int age;
        Date date;
        Division div(divName);
        vector<JobDescription> tempJobs;
```

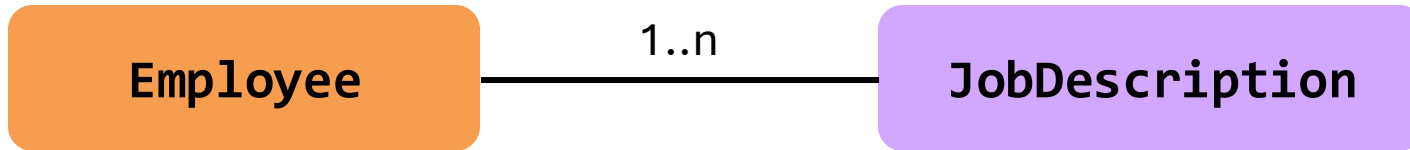
Хэрэгжүүлэлт: Division



```
class Employee : public Person {  
    private:  
        Division division;
```

```
cout << "Хэлтэс: "; cin >> ws; getline(cin, divName);  
Division div(divName);
```

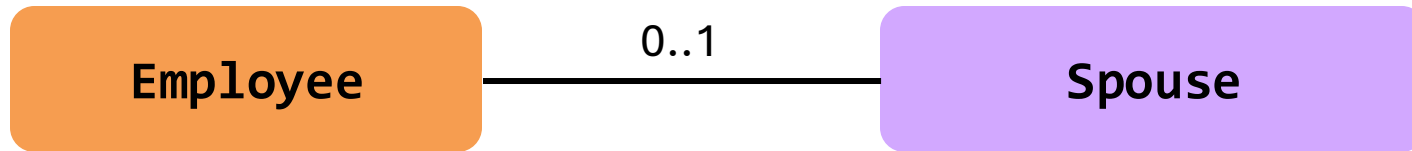
Хэрэгжүүлэлт: JobDescription



```
class Employee : public Person {
    private:
        vector<JobDescription> jobDescriptions;

    cout << "Хэдэн ажлын тодорхойлолт нэмэх вэ?: "; cin >> jobCount;
    if (jobCount < 1) jobCount = 1; // 1..n шаардлага хангах
    for (int i = 0; i < jobCount; i++) {
        string jobName;
        cout << i + 1 << "-р мэргэжил: ";
        cin >> ws; getline(cin, jobName);
        tempJobs.push_back(JobDescription(jobName));
    }
```

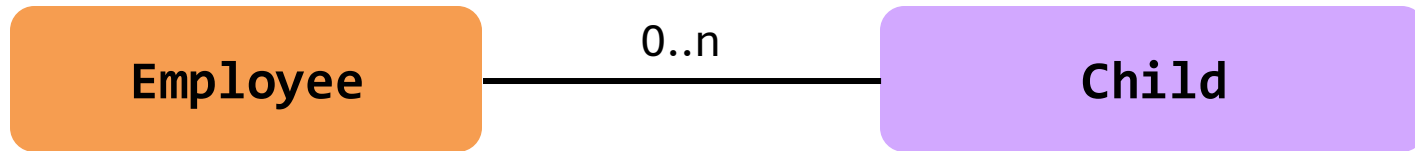
Хэрэгжүүлэлт: Spouse



```
class Employee : public Person {  
    private:  
        Spouse spouse;
```

```
char hasSpouse;  
Spouse spouse;  
cout << "Гэр бүлийн хүний мэдээлэл бүртгэх үү? (y/n): "; cin >> hasSpouse;  
if (hasSpouse == 'y' || hasSpouse == 'Y') { ... }
```

Хэрэгжүүлэлт: Child



```
class Employee : public Person {
private:
    vector<Child> children;
```

```
int childCount;
vector<Child> tempChildren;
cout << "Хэдэн хүүхэдтэй вэ? (Байхгүй бол 0): "; cin >> childCount;
for (int i = 0; i < childCount; i++)
{
    ...
    tempChildren.push_back(child);
}
```


Хэрэгжүүлэлт: printObj

```
#include "printObj.h"

void printObj(const vector<Employee> &employees)
{
    for (const auto& employee : employees) {
        employee.print();
    }
}
```

Хэрэгжүүлэлт: main

```
#include "oop.h"

int main() {
    vector<Employee> staff = setEmployees();
    printObj(staff);

    cout << "Программ дууслаа." << endl;
    return 0;
}
```

Хэрэгжүүлэлт:
Программын
гаралт

=====

АЖИЛТНЫ ДЭЛГЭРЭНГҮЙ ХУУДАС

=====

Нэр: Баттамир
Регистрийн дугаар: УП06212577
Нас: 20
Ажилтны ID: 24B1NUM0616
Албан тушаал: Суралцагч
Ажилд орсон: 2024/9/2
Хэлтэс: МТЭС

--- Гэр бүлийн хүн ---

Нэр:
Регистрийн дугаар:
Нас: 0
Гэрлэсэн огноо: 0/0/0

--- Ажлын тодорхойлолтууд ---

Мэргэжил: Компьютерын ухаан
Мэргэжил: Программ хангамж

--- Хүүхдүүд ---

Нэр:
Регистрийн дугаар:
Нас: 0
Дуртай тоглоом:

=====

Программ дууслаа.

Дүгнэлт

Энэхүү лабораторийн ажлаар OOP-ын гол зарчмуудыг Employee классыг жишээ болгон судалсан. Удамшил нь "is-a" харилцааг илэрхийлж, кодын ахин ашиглалт болон олон хэлбэржилтийг дэмждэг. Бүрдэл нь эцэг объект устахад хүүхэд объектууд хамт устдаг хатуу холболт бол нэгтгэл нь объектууд бие даан оршин тогтнох боломжтой сул холболт юм. Employee класс нь Person-оос удамшиж, Spouse, Division, Child зэрэг класстай харилцаа үүсгэснээр эдгээр ойлголтуудыг практикт хэрэгжүүлэх чадварыг бататгалаа.

Эх сурвалж

1. М. Золжаргал багшийн лекцийн материал
2. Объект хандлагат технологийн С++ программчал, Ж. Пүрэв, 2008, Улаанбаатар
3. [geeksforgeeks](http://www.geeksforgeeks.com)

**Анхаарал хандуулсанд
баярлалаа**